

Ublox Local Persistence - Complete Solution

Overview

Answer to your question: *Yes, local environment can absolutely be used for database persistence!*

The `ublox_persistence.py` system provides complete local file-based storage with **no database server required**. Everything persists to `~/.ublox/` directory.

Storage Strategy

1. SQLite Database (40 KB)

- Path: `~/.ublox/ublox.db`
- Stores: Players, PTOs, PTS tokens, transactions, play sessions
- No server needed - just a local file
- ACID compliant, lightweight, zero configuration

2. Pickle Files (1.8 KB each)

- Path: `~/.ublox/cdt_states/*.pkl`
- Stores: CDT numpy arrays (coherence fields)
- Reason: numpy arrays don't fit well in SQL
- One file per CDT

3. JSON Export (1.7 KB)

- Path: `~/.ublox/export.json`
- Complete data export for portability
- Human-readable backup format

4. Backups

- Path: `~/.ublox/backups/{timestamp}/`
- Automatic timestamped snapshots
- Complete directory copies

What Gets Persisted

Players

```
{
  "player_id": "alice",
  "tau_k": 8.5,                # Temporal coherence coefficient
  "usd_obbba_balance": 5000.0, # Investment capital
  "xUSD_balance": 0.0,        # Composed coherence value
}
```

```

"total_play_time": 0.0,           # Hours in presence
"quality_score": 0.0,            #  $\tau_k$  quality
"phase_locks_count": 0,          # Synchronization events
"memory_depth": 0.0              # Accumulated phase-locks
}

```

PTOs (Public Time Offerings)

```

{
  "pto_id": "pto_harmonic_001",
  "game_name": "Harmonic Caverns",
  "creator_id": "alice",
  "atu_required": 500,            # Agential Time Units needed
  "pts_offered": 2000,           # Project Time Shares for sale
  "funding_raised": 0.0,         # USD-OBBA raised
  "status": "funding",           # funding/in_progress/completed
  "aci_coherence_score": 0.0,    # ACI evaluation (0-1)
  "xUSD_pool": 0.0,              # Total xUSD composed
  "world_cdt_id": "pto_harmonic_001_world_cdt" # Link to CDT file
}

```

PTS Tokens (Project Time Shares)

```

{
  "token_id": "pts_001",
  "game_id": "pto_harmonic_001",
  "holder": "alice",
  "hours_committed": 40.0,       # Time investment
  "purchase_price": 1000.0,      # USD-OBBA paid
  "hours_played": 0.0,           # Time in coherent presence
  "xUSD_accumulated": 0.0,       # Value composed through play
  "tau_k_at_purchase": 8.5       #  $\tau_k$  when purchased
}

```

Transactions

```

{
  "tx_id": 1,
  "tx_type": "investment",        # investment/pto_completion/etc
  "from_player": "alice",
  "to_player": "pto_harmonic_001",
  "amount": 1000.0,
  "currency": "USD-OBBA",
  "pto_id": "pto_harmonic_001",
  "timestamp": "2025-12-12 16:56:32"
}

```

Play Sessions

```
{
  "session_id": 1,
  "pto_id": "pto_harmonic_001",
  "player_id": "alice",
  "duration": 5.0,           # Hours in presence
  "quality": 0.85,          #  $\tau_k$  coherence
  "xUSD_earned": 425.0,      # Value composed in daThiccNOW
  "timestamp": "2025-12-12 16:56:32"
}
```

Database Schema

Tables

players

- player_id (PRIMARY KEY)
- position_x, position_y, phase, coherence
- tau_k, resonance_freq
- usd_obbba_balance, xUSD_balance
- total_play_time, quality_score
- phase_locks_count, memory_depth
- created_at

ptos

- pto_id (PRIMARY KEY)
- game_name, creator_id (FK), description
- atu_required, atu_spent
- pts_offered, pts_sold, price_per_pts
- funding_goal, funding_raised
- status, launch_time, funding_deadline, completion_time
- aci_coherence_score, xUSD_pool
- world_cdt_id (link to pickle file)

pts_tokens

- token_id (PRIMARY KEY)
- game_id (FK), holder (FK)
- hours_committed, hours_played
- purchase_price, xUSD_accumulated
- tau_k_at_purchase, purchase_time

transactions

- tx_id (AUTOINCREMENT PRIMARY KEY)
- tx_type, from_player, to_player
- amount, currency
- pto_id (FK)
- timestamp, metadata

play_sessions

- session_id (AUTOINCREMENT PRIMARY KEY)
- pto_id (FK), player_id (FK)
- duration, quality, xUSD_earned
- timestamp

Usage

Basic Save/Load

```
from ublox_persistence import UbloxLocalStorage

# Initialize
storage = UbloxLocalStorage() # Defaults to ~/.ublox/

# Save player
storage.save_player(player)

# Load player
player_data = storage.load_player("alice")

# Save PTO (automatically saves CDT to pickle)
storage.save_pto(pto)

# Load PTO (automatically loads CDT)
pto_data = storage.load_pto("pto_001")

# Log transaction
storage.log_transaction(
    tx_type="investment",
    from_player="alice",
    to_player="pto_001",
    amount=1000.0,
    currency="USD-0BBBA"
)

# Log play session
storage.log_play_session(
    pto_id="pto_001",
    player_id="alice",
    duration=5.0,
    quality=0.85,
    xUSD_earned=425.0
```

```
)
```

Backups

```
# Create backup
backup_path = storage.create_backup("before_pto_completion")

# List backups
backups = storage.list_backups()

# Restore from backup
storage.restore_backup("backup_20251212_112719")
```

Analytics

```
# Global stats
stats = storage.get_global_stats()
# Returns: {players: {...}, ptos: {...}, pts_tokens: {...}, transactions: {...}}

# Export to JSON
storage.export_to_json() # Creates ~/.ublox/export.json
```

Integration with PTO Engine

The `PersistentUbloxEngine` class (in `ublox_persistent_demo.py`) shows how to integrate:

```
class PersistentUbloxEngine(UbloxPTOEngine):
    def __init__(self, persist=True):
        super().__init__()
        self.storage = UbloxLocalStorage() if persist else None

    def launch_pto(self, ...):
        pto = super().launch_pto(...)
        if self.persist_enabled:
            self.storage.save_pto(pto) # Auto-save!
        return pto
```

Every operation auto-saves to disk. No manual persistence needed!

Storage Size

For a typical PTO with 3-5 players:

- SQLite database: ~40 KB
- CDT pickle files: ~2 KB each
- Total: < 100 KB

Extremely lightweight!

Philosophical Note: Composition vs. Earning

In FHP paradigm, xUSD is not "earned" over time but **composed in the present moment**.

- Traditional: `xUSD = hours_worked × rate` (linear accumulation)
- FHP: `xUSD = presence_coherence × daThiccNOW` (compositional amplification)

Each moment of coherent presence **composes** value by amplifying the temporal density of the present. This is stored as `xUSD_accumulated` but represents **compositional intensity** not accumulated time.

Key Insights

1. **No Server Required:** SQLite is just a file, runs in-process
2. **ACID Compliance:** SQLite guarantees transactional integrity
3. **Zero Configuration:** Just works out of the box
4. **Portable:** Everything in one directory, easy to backup/migrate
5. **Human Readable:** JSON export for inspection
6. **Hybrid Approach:** SQL for relational data, Pickle for numpy arrays, JSON for export

Demo Results

Run `python3 demo_persistence_simple.py` to see:

-  Player saved to SQLite
-  PTO saved to SQLite + CDT to pickle
-  PTS token saved
-  Transaction logged
-  Play session logged
-  All data verified by loading back
-  Backup created
-  JSON export created

Total storage: < 100 KB

Files Created

```
~/ublox/
├─ ublox.db                # SQLite database (40 KB)
├─ export.json             # JSON export (1.7 KB)
├─ cdt_states/             # CDT pickle files
│   └─ pto_harmonic_001_world_cdt.pkl (1.8 KB)
├─ backups/                # Timestamped backups
│   └─ demo_backup/
│       ├── ublox.db
│       └─ cdt_states/
│           └─ pto_harmonic_001_world_cdt.pkl
```

Conclusion

✔ Yes, local environment works perfectly for database persistence.

The hybrid approach (SQLite + Pickle + JSON) provides:

- **Lightweight:** < 100 KB per PTO
- **Fast:** In-process, no network latency
- **Reliable:** ACID compliant
- **Portable:** One directory, easy to move
- **Zero Setup:** No server installation needed

Perfect for development, testing, and even production for single-user or small-scale deployments. Can scale to multi-user by replacing SQLite with PostgreSQL/MySQL later without changing the API.

"Memory is not stored bits, but accumulated phase-locks.

Value is not earned over time, but composed in daThiccNOW."

— Fractal Harmonic Principle